On the Geometry of Reinforcement Learning in Continuous State and Action Spaces

Saket Tiwari
Department of Computer Science
Brown University
Providence, RI 02906
saket_tiwari@brown.edu

Omer Gottesman
Department of Computer Science
Brown University
Providence, RI 02906

George Konidaris
Department of Computer Science
Brown University
Providence, RI 02906

Abstract

Advances in reinforcement learning have led to its successful application in complex tasks with continuous state and action spaces. Despite these advances in practice, most theoretical work pertains to finite state and action spaces. We propose building a theoretical understanding of continuous state and action spaces by employing a geometric lens. Central to our work is the idea that the transition dynamics induce a low dimensional manifold of reachable states embedded in the high-dimensional nominal state space. We prove that, under certain conditions, the dimensionality of this manifold is at most the dimensionality of the action space plus one. This is the first result of its kind. We then incorporate environment dynamics into the geometry of the state space, which allows us to formulate reinforcement learning as a continuous-time stochastic process on a manifold. Finally, we prove that a randomly initialized deep policy defines is a diffusion process on the manifold, in the infinite width limit. Our results are for the deep policy gradient setting where the policy is parametrized by a deep neural network. This has implications on the distribution of states attained by a randomly initialized agent. We provide empirical results that corroborate our theory.

1 Introduction

A reinforcement learning (RL) agent learns by interacting with an environment described by a state embedded in a state space, using a set of available actions. Recent advances in RL research have lead to agents being able to successfully learn in environments with enormous state spaces. Some examples are games [Mnih et al., 2015, Silver et al., 2016], robotic control in simulated [Lillicrap et al., 2015, Schulman et al., 2015, 2017] and real environments [Levine et al., 2016, Zhu et al., 2020, Deisenroth and Rasmussen, 2011]. However, we do not have the necessary theoretical tools to understand why these approaches work so well in practice.

We propose to study the dynamics of reinforcement learning agents in continuous state and action spaces through the lens of geometry. We build on the intuition behind the *manifold hypothesis*, which states that most high-dimensional real-world data sets actually lie on low-dimensional manifolds [Tenenbaum, 1997, Carlsson et al., 2007, Fefferman et al., 2013, Bronstein et al., 2021]; for example, the set of natural images are a very small, smoothly-varying subset of all possible images.

For the longest time, researchers have utilised the manifold hypothesis in RL—i.e. hypothesised or made assumptions about the effective state space lying on a low dimensional manifold [Mahadevan, 2005, Machado et al., 2017, 2018, Banijamali et al., 2018, Wu et al., 2019, Liu et al., 2021]. One of the achievements this has lead to is improved performance on long horizon tasks, such as Montezuma's revenge, by designing intrinsic rewards [Machado et al., 2017]. Our main result provides a general proof of this hypothesis for *all* continuous RL environments, and to the best of our knowledge is the first such result of its kind. Furthermore, we provide on upper bound on the dimensionality of this manifold, and provide intuition as to how it emerges. Concretely, we prove that the dimensionality of the state manifold is at most the dimensionality of the action space plus one, for deterministic environments with continuous states and actions.

We empirically corroborate this result by estimating the dimensionality in multiple MuJoCo environments using nearest neighbour estimation [Facco et al., 2017]. Our empirical results suggest that in many instances the bound on the dimensionality of the manifold is tight.

In most popular deep RL algorithms for continuous control, the agent's policy is parametrized by a DNN [Lillicrap et al., 2015, Schulman et al., 2015, 2017] initialized with random weights, and with independent Gaussian noise added to its output to improve exploration. We define a random variable corresponding to the continuous time trajectory sampled by a RL agent with additive noise, and prove that this is in fact a *diffusion process* on the state manifold in the infinite width limit of the policy DNN. We later discuss why this is a valid limit and is close practice. This also allows us to apply policy networks for sampling data on the state manifold for the estimation of manifold dimension in a theoretically sound manner.

2 Background and Mathematical Preliminaries

2.1 Continuous-Time Reinforcement Learning

We analyze the setting of continuous-time reinforcement learning in a deterministic *Markov decision process* (MDP) which is defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, f, f_r, s_0, \lambda)$ over time $t \in [0, T)$. $\mathcal{S} \subset \mathbb{R}^{d_s}$ is the set of all possible states of the environment. $\mathcal{A} \subset \mathbb{R}^{d_a}$ is the compact set of actions available to the agent that it can take at all times. $f : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^+ \to \mathcal{S}$ and $f \in C^\infty$ is a *smooth* function that determines the state transitions: $s' = f(s, a, \tau)$ is the state the agent transitions to when it takes the action a at state s for the time period τ. Note that $f(s, a, 0) = s$, meaning that the agent's state remains unchanged if an action is applied for a duration of $\tau = 0$. The reward obtained for reaching state s is $f_r(s)$, determined by the reward function $f_r : \mathcal{S} \to \mathbb{R}$. s_t denotes the state the agent is at time t and a_t is the action it takes at time t. s_0 is the fixed initial state of the agent at $t = 0$, and the MDP terminates at $t = T$. The agent does not have access to the functions f and r. It can only observes states and rewards at all times $t \in [0, T)$.

The agent is equipped with a policy, $\pi : \mathcal{S} \to \mathcal{A}$, that determines its decision making process. We denote the set of all the possible policies by Π. Simply put, the agent takes action $\pi(s)$ at state s. The goal of the agent is to maximise the discounted return $J(\pi) = \int_0^T e^{-\frac{l}{\lambda}} r(s_l) dl$, where $s_{t+\epsilon} = f(s_t, \pi(s_t), \epsilon)$ for infinitesimally small ϵ and all $t \in [0, T)$. We define the *action bundle*, $g : \mathcal{S} \times \mathcal{A} \to \mathbb{R}^{d_s}$, for an MDP as

$$g(s, a) = \lim_{\epsilon \to 0} \frac{f(s, a, \epsilon) - s}{\epsilon} = \frac{\partial f(s, a, \epsilon)}{\partial \epsilon}.$$

Intuitively, this captures the direction in which the agents state changes at state s upon taking an action a. For notational convenience we will denote $g(s, \pi(s))$ as $g_\pi : \mathcal{S} \to \mathbb{R}^{d_s}$ and name it the *action flow* of the environment defined for a policy π. Note that g_π is a well defined function. Intuitively, g_π is the direction of change in the agent's state upon following a policy π at state s for an infinitesimally small time. This solution is also unique and can be written in the integral form as

$$s_t^\pi = s_0 + \int_0^t g_\pi(s_l^\pi) dl. \tag{1}$$

The above curve is a smooth curve if the policy is also smooth. Therefore, given an MDP, $\mathcal{M}$, and a smooth deterministic policy, $\pi \in \Pi$, the agent traverses a continuous time trajectory or curve $H_{\mathcal{M},\pi} : [0, T) \to \mathcal{S}$.

We also define the value function at time t for a policy π as the cumulative future reward starting at time t.

$$v^{\pi}(s_t) = \int_t^T e^{-\frac{l-t}{\lambda}} r(s_l) dl. \tag{2}$$

Note that the objective function, $J(\pi)$, is the same as $v^{\pi}(s_0)$. Our specification is very similar to classical control and continuous time RL [Cybenko, 1989, Doya, 2000] with the key difference being how we define the transition function, f.

2.2 Manifolds

The MDPs that RL agents are applied to often have a low dimensional structure to their state space, resulting in it having fewer degrees of freedom than its nominal dimesionality. For example, in the case of learning from visual input in a MuJoCo environment like 2D cheetah, one can describe the cheetah's state by its "pose" and position instead of the 128×128 pixels of the image. The idea of a low dimensional manifold embedded in a high dimensional state space formalises this notion.

A function $h : X \to Y$, from one open subset $X \subset \mathbb{R}^{l_1}$, to another open subset $Y \subset R^{l_2}$, is a diffeomorphism if h is bijective, and both h and h^{-1} are differentiable. Intuitively, a low dimensional surface embedded in a high dimensional Euclidean space can be parametrized by a differentiable mapping, and if this mapping is bijective we term it a diffeomorphism. Here X is said to be diffeomorphic to Y. A manifold is defined as follows:

Definition 2.1. *Let $k, m \in \mathbb{N}_0$. A subset $M \subset \mathbb{R}^k$ is called a smooth m-dimensional submanifold of $\mathbb{R}^k$ (or m-manifold in $\mathbb{R}^k$) iff every point $p \in M$ has an open neighborhood $U \subset \mathbb{R}^k$ such that $U \cap M$ is diffeomorphic to an open subset $O \subset \mathbb{R}^m$. A diffeomorphism, $\phi : U \cap M \to O$ is called a coordinate chart of M and the inverse, $\psi := \phi^{-1} : O \to U \cap M$ is called a smooth parametrization of $U \cap M$.*

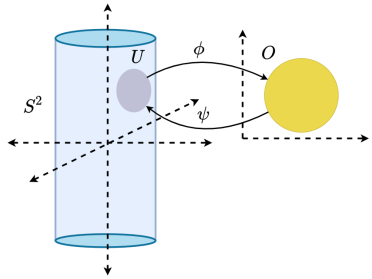


Figure 1: The surface of an open cylinder of unit radius, denoted by S^2, in $\mathbb{R}^3$ is a 2D manifold embedded in a 3D space. More formally, $S^2 = \{(x, y, z) | x^2 + y^2 = 1, z \in (-h, h)\}$ where the cylinder's height is $2h$. One can smoothly parametrize S^2 as $\psi(\theta, b) = (\sin\theta, \cos\theta, b)$. The coordinate chart is $\phi(x, y, z) = (\sin^{-1} x, z)$.

We illustrate this with an example in Figure 1. If $M \subset \mathbb{R}^k$ is a non-empty smooth m-manifold then $m \leq k$. This reflects the idea that a manifold is of lower or equal dimension than its ambient dimension. A smooth curve $\gamma : I \to M$ is defined from an interval $I \subset \mathbb{R}$ to the manifold M as a function that is infinitely differentiable for all t. The derivative of γ at t is denoted as $\dot{\gamma}(t)$. The tangent space is defined as follows.

Definition 2.2. *Let M be an m-manifold in $\mathbb{R}^k$ and $p \in M$ be a fixed point. A vector $v \in \mathbb{R}^k$ is called a tangent vector of M at p if there exists a smooth curve $\gamma : I \to M$ such that $\gamma(0) = p, \dot{\gamma}(0) = v$. The set*

$$T_pM := \{\dot{\gamma}(0) | \gamma : \mathbb{R} \to M \text{ is smooth}, \gamma(0) = p\}$$

of tangent vectors of M at p is called the tangent space of M at p.

Continuing our example, the tangent space of a point p in S^2 is the vertical plane tangent to the cylinder at that point. For a small enough ϵ and a vector $v \in T_pS^2$ there exists a unique curve $\gamma : [-\epsilon, \epsilon] \to S^2$ such that $\gamma(0) = p$ and $\dot{\gamma}(0) = v$.

3 State Space Geometry

The state space is typically thought of as a dense space in which all states lie, but it is not necessarily the case that all such states are reachable by the agent. Indeed, in examples such as the Cheetah environment discussed in Section 2.2, the vast majority of images in the ambient 128x128 state space will never be observed because they do not correspond to any image of a Cheetah, or even any recognizable image at all. Instead, the agent must be traversing through a much smaller subset of this space as it interacts with the world. Our goal is to characterize the geometry of this space. Two main factors constrain the states available to an agent: **1)** the transition function and the actions that are available to an agent, and **2)** the start state s_0. We therefore define $\mathcal{S}_e$ as the *effective set of states*. Under the assumptions that Π is the set of all smooth policies, $\pi : \mathcal{S} \to \mathcal{A}$, for a continuous time MDP, $\mathcal{M}$, and those made in section 2.1 we define the effective set of states as follows.

Definition 3.1. *For an MDP, $\mathcal{M}$, the effective set of states, $\mathcal{S}_e$, is defined as the union of the sets of states of all possible continuous curves traversed by the agent for the set of all smooth policies. Formally, $\mathcal{S}_e = \cup_{\pi \in \Pi}\{s | s = H_{M,\pi}(t)$ for some $t \in [0, T)\}$, where Π is the set of all almost everywhere (a.e.) smooth policies.*

We make two additional assumptions:

1. **No orbits:** for any policy $\pi \in \Pi$ we do not have any critical points or loops in the differential field defined by g_π. Formally, we have the following conditions: $g_\pi(s) \neq \mathbf{0}$ for all $s \in \mathcal{S}$, and for a fixed policy π the solution for the equation $s_t^\pi = s$, if it exists, is unique in t.

2. **Action equivalence condition:** There exists an $r > 0$ such that for all $0 < \epsilon < r$, if we have $f(s, a_1, \epsilon) = f(s, a_2, \epsilon)$ for some fixed $a_1, a_2 \in \mathcal{A}$ and any one $s \in \mathcal{S}$ then it holds for all $\mathcal{S}$.

Under the continuous RL setting described in section 2.1 and these two assumptions our main result is:

Theorem 3.2. *The set of effective states, $\mathcal{S}_e$, is a smooth manifold of dimensionality at most $d_a + 1$.*

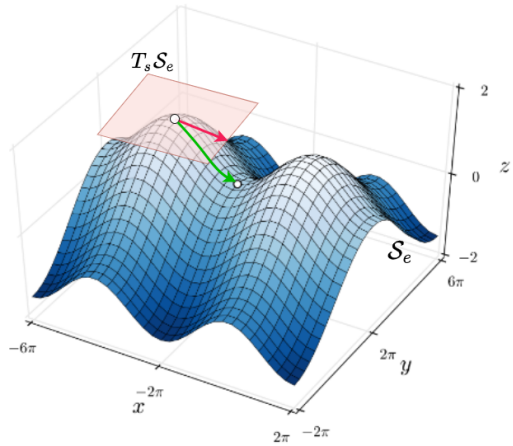
We provide the proof in Appendix B. Intuitively, the action set limits the directions in which the agent can go. For example, a single action executed for a finite time interval makes the agent traverse a 1D curve. The union of all such curves is the only subset of the state space the agent can reach. The key technical idea is to construct a coordinate chart, and therefore a diffeomorphism, from a subset of low dimensional euclidean space to a subset of the state space. We fix a state s and consider the function $\phi : \mathcal{A} \times (0, r) \to \mathcal{S}$ such that $\phi(a, \epsilon) = f(s, a, \epsilon)$. This result pertains to the *global geometry* of the state manifold, meaning the state manifold everywhere has this low dimensional structure.

Our result formalises the long-held idea that there is in fact a lower dimensional structure to the state manifold; in particular, this is true when $d_a + 1 < d_s$, which is almost always the case for RL environments. It also formalises the idea that the agent can only observe data for states reachable by interacting with the environment using the set of actions at its disposal. Henceforth, we refer to $\mathcal{S}_e$ as the state manifold.

We also present an immediate corollary of Theorem 3.2.

Corollary 3.3. *For every $a \in \mathcal{A}$ and fixed $s \in \mathcal{S}_e$, the action bundle, $g(s, a)$, maps from the set of actions $\mathcal{A}$ to $T_s\mathcal{S}_e$ and therefore there exists a vector $v_a \in T_s\mathcal{S}_e$ such that $v_a = g(s, a)$ for all $a \in \mathcal{A}$.*

This implies that locally an agent is moving along a tangent in the tangent space of a state in the state manifold as it acts upon the environment. The corollary follows from Definition 2.2, and its proof is provided in Appendix D. Intuitively, Corollary 3.3 implies that locally the agent's state changes in a constrained manner dictated by the actions available to it and the *local geometry* of the manifold, i.e., the tangent space and its projection onto the state manifold. We illustrate this in Figure 2. We note that the idea of actions mapping to the tangent space was assumed earlier by Liu et al. [2021] for constrained RL but Corollary 3.3 both proves this assumption, as well as show that it holds in general for all continuous RL environments.



r

Figure 2: Consider a hypothetical scenario where, for some policy, π, even though the agent moves locally at state s in the direction of the red arrow (along $g_\pi(s)$) as time progresses it ends up travelling along the green arrow which is on the manifold $\mathcal{S}_e$. This hypothetical constraint restricts the state space "available" to an agent. Thus, the agent can only reach the states that have a valid solution for the Equation 1 for any smooth policy .

4 Reinforcement Learning as a Stochastic Process

4.1 Stochastic Processes

Up to this point we have discussed the the manifold of reachable states of an MDP. This manifold is determined by the union of all reachable states for all deterministic policies. We now turn to consider stochastic policies and show that such stochastic policies determine a state distribution over the manifold. In order to formalise the properties of this distribution, we describe the agent as a stochastic process over the environment. These results, describing not only what states are *theoretically* reachable, but what states are likely to be reached, can give insight into the nature of exploration in continuous spaces.

We are interested in "random curves" in addition to the single instance of a curve γ described above. The theory of stochastic processes deal with the formalism of random curves or *processes*. We first provide definitions from probability theory that are used in developing the aforementioned formalism. A stochastic process on a manifold, M, is a time dependent random variable, η, given on a probability space, $(\Omega, \mathcal{F}, P)$, and taking values in M that depends on time $t \in [0, T)$ where $T < \infty$. A process $\eta(t), t$, has almost surely continuous sample paths if for almost everywhere $\omega \in \Omega$ the curve $\eta(t, \omega) \to M$ is continuous in t. We abuse notation to denote a single instance of a random curve $\eta(t)$ using $\eta(t, \omega)$.

A one dimensional Wiener process [Durrett, 1984], taking values in the Euclidean space $\mathbb{R}$, is one of the building blocks of the theory of stochastic processes.

Definition 4.1. *A stochastic process $w(t)$ is called a Wiener process if the sample paths of $w(t)$ are almost surely continuous, square integrable Martingale with $w(0) = 0$, and $E[(w(t) - w(s))^2)] = t - s$ for $t \geq s$.*

The multidimensional Wiener process is a concatenation of such single dimensional processes. We now define the Itô integral which is applied to integrate random variables that vary with time. Let A be a random linear operator going from $\mathbb{R}^k$ to $\mathbb{R}^n$. For a partition of the time interval $q = (0 = t_0 < t_1 < t_2 ... < t_q = T)$ consider the integral sum $\sum_{i=0}^{q-1} \frac{A_{i+1}+A_i}{2}(w(t_{i+1}) - w(t_i))$, where $w_k(t)$ is a Wiener process in $\mathbb{R}^k$. As $diam(q) \to 0$ the limiting sum is defined to be the Stratonovich integral (see section 6.2.2 by [Gliklikh, 2010] for different types of stochastic integrals). A stochastic process, $\eta(t)$, on a manifold M is defined by the integral equation

$$\eta(t) = \eta(0) + \int_0^t a(\eta(t))dl + \int_0^t A(\eta(t)) \circ dw_k(l),$$

where $a(p)$ is a vector field on the manifold M with bounded variation and $A(p)$ is a linear operator from $\mathbb{R}^k$ to T_pM, $p \in M$. The first integral in the summation on the right hand side of the equality is a Lebesgue integral and the second one is a Stratonovich integral. Here, the stochastic process is on a manifold embedded in some Euclidean space, $\mathbb{R}^k$. Note that such a process $\eta(t)$ is a diffusion process on manifold M. Although $\eta(t) \in \mathbb{R}^k$, it can be shown that if $\eta(0)$ lies on the manifold $M \subset \mathbb{R}^k$ then the solutions $\eta(t)$ remain on the manifold M at all times (pages 22-23 in the textbook by Hsu [2002]). We provide further background for stochastic processes on manifolds in Appendix C.

4.2 A Stochastic Process Description of Reinforcement Learning

Fully connected DNNs with ReLU activation function have been applied to reinforcement learning in various continuous state and action spaces [Lillicrap et al., 2016, Schulman et al., 2015, 2017]; these are piece-wise linear with a boundary at 0. We will instead be replacing ReLU with Gaussian Error Linear Units (GELU) Hendrycks and Gimpel [2016] these are a smooth approximation of the ReLU activation. DNNs constructed using only GELU activation are smooth in the input space and thereby staisfy our assumption in Section 2.1, that the family of policies is smooth. We assume that these networks have a single hidden layer of width n. More formally, for any input $x \in \mathbb{R}^{n_{\text{in}}}$ we have

$$z(x) = Ux, \;\; h(x) = \varphi(z(x)), \;\; F(x) = Vh(x),$$

where W is an $n_{\text{in}} \times n$ matrix, φ is GELU activation and V is an $n \times n_{\text{out}}$ matrix. This defines a single hidden layer DNN with output dimension n_{out} and no biases. We denote by $\theta = \{U, V\}$ the set of parameters for F. In our simplified setting, a DRL agent parametrises the policy π with a DNN, F^π, with parameters θ_π and the value function, F^v, with another DNN with parameters θ_v. The parameters, θ_π and θ_v, of these neural networks are initialized randomly as done in the *maximal update parametrization* (MUP) introduced by Yang and Hu [2020]: $U = \sqrt{n}u, V = \frac{1}{\sqrt{nv}}$ and with initialization $u_{i,j}, v_{i,j} \sim \mathcal{N}(0, 1)$.

We consider the deep deterministic policy gradient (DDPG) [Lillicrap et al., 2015] algorithm. In the very first stage (or epoch) of learning, the algorithm proceeds by performing rollouts of a randomly initialised policy with additive Gaussian noise. Formally, for any state $s \in \mathcal{S}_e$ the agent takes an action $\pi(s) = F^\pi(s) + \mathcal{N}(0, \sigma_\pi)$. Applying this to our theoretical setting of continuous time RL, these rollouts are random trajectories on the effective state manifold $\mathcal{S}_e$ and the state of the agent at time t is a random variable, denoted by S_t, which is supported on $\mathcal{S}_e$. To study these random trajectories we employ the notion of stochastic processes as noted above. We make one further assumption, in addition to the ones made in sections 2 and 3, that action bundle the function, $g(s, a)$ is linear in a. Therefore, it can be represented as

$$g(s, a) = b_s + A_s a, \tag{3}$$

where $b_s \in T_s\mathcal{S}_e$ and A_s is a $d_s \times d_a$ matrix. Under the assumptions on initialisation and of Sections 2.1 and 3 we have the following result.

Theorem 4.2. *Let S_t denote the state random variable at time t in a continuous time MDP for a randomly initialised, as in the maximal update parametrization, policy with single hidden layer fully connected DNN of width n with GELU activation and additive Gaussian noise. Let the continuous time MDP have a start state s_0, action bundle g that is linear in a. The stochastic differential equation for S_t, in Stratonovich formulation,*

$$dS_t = b_s dt + A_s \circ dw_{d_a}(t),$$

where b_s and A_s are as in Equation 3, converges to a unique solution in distribution as $n \to \infty$

$$S_t = s_0 + \int_0^t b_s dl + \int_0^t A_s \circ dw_{d_a}(l), \tag{4}$$

where the second integral is a Stratonovich integral. Additionally, it is a diffusion process on the manifold $\mathcal{S}_e$.

This theorem follows from MUP, linearity of $g(s, .)$ and the definition of a diffusion process. Proof is provided in Appendix D due to space constraints. As discussed in Section 4, S_t is in fact a random variable that remains on the manifold $\mathcal{S}_e$. Therefore the main takeaway from this theorem is that a randomly initialized policy, as described above, induces a well-defined probability distribution characterized by the diffusion process on the state manifold. Samples taken from the rollouts of such a policy are in fact samples on the manifold.

5 Discrete Time Reinforcement Learning

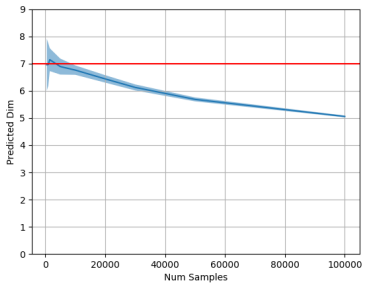
We have presented our results in the continuous time RL setting which is an underutilized theoretical tool in the study of RL. Here, and in the experimental section, we argue that it is in fact a useful model for theoretical analysis in the continuous state and action, but discrete time, setting. The primary intuition is that, in context of simulated robotic control problems, our work approximates the agents behavior during the transition period $(t, t+1)$ as the action a_t changing to a_{t+1} in a smooth manner and similarly for s_t to s_{t+1}. In this context, the discrete-time observations can be viewed as time-uniform *samples* from an underlying continuous process. Since the stochastic process on the effective state manifold reasons about random curves as the primary object of study we argue that these discrete trajectories can be *augmented* to obtain continuous curves.

More formally, if we consider a discrete time MDP with continuous state and action spaces a discrete state trajectory is the sequence of states $\{s_t\}_{t=1}^T$ and similarly for actions, $\{a_t\}_{t=1}^T$. We augment the discrete time action trajectories with a function for the transition period $\vartheta : \mathcal{A} \times [0,1] \times \mathcal{A} \to \mathcal{A}$ we can construct a model for these smooth transitions of the action. This function ϑ is such that $\vartheta(a_t, l, a_{t+1}) = a_{t+l}$ for $l \in [0,1], \forall t \in \{0, 1, ..., T-1\}$, the boundary conditions hold, meaning $\vartheta(a_t, 0, a_{t+1}) = a_t$ and $\vartheta(a_t, 1, a_{t+1}) = a_{t+1}$, and it is smooth in the transition time l. Note that we can also construct this ϑ such that it is differentiable at a_t for $\vartheta(a_{t-1}, l, a_t)$ and $\vartheta(a_t, 1-l, a_{t+1})$ as $l \to 1^-$. More importantly, by extending continuous time results to discrete time results we implicitly assume that there exists a function f that preserves smoothness of the state transitions. Specifically, given $a_{t+l} = \vartheta(a_t, l, a_{t+1})$ there exists a transition function f and consequently the action bundle g, as defined in Section 2, such that $s_t + \int_0^1 g(s_t, \vartheta(a_t, l, a_{t+1}))dl$ equals s_{t+1}. The proof of existence of such a function, ϑ, is beyond the scope of our current work. Such an augmentation transforms all trajectories on this discrete time MDP to continuous time smooth curves. As in many other fields of engineering, we argue that we can only obtain measurements for various signals at discrete intervals. For example, for a robot with various joints we can only measure the joints over intervals which are then attributed to discrete time measurements. Similarly, we can actuate the motors up to a frequency and those can be considered as discrete control signals, even though the underlying physical process is continuous time.

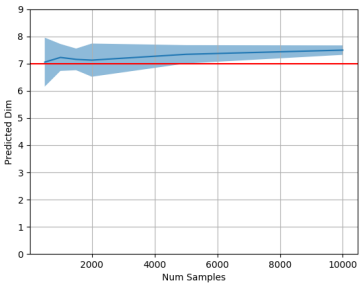
6 Related Work

There has been significant empirical work that assumes the state space to be a manifold in RL. The primary approach has been to study discrete state spaces as data lying on a graph which has an underlying manifold structure. Mahadevan [2005] provided the first such framework to utilise the manifold structure of the state space in order to learn value functions. This was followed by their work explaining proto value functions (PVF) in great detail and applying them to various toy problems [Mahadevan and Maggioni, 2007]. Machado et al. [2017] showed that PVFs implicitly define options and applied them to high dimensional discrete action MDPs (Atari games). Wu et al. [2019] provided an overview of varying geometric perspectives of the state space in RL and also show how the graph Laplacian is applied to learning in RL. Another line of work, that assumes the state space is a manifold, is focused on learning manifold embeddings or mappings. Bush and Pineau [2009] apply manifold embeddings to reconstruct observable state space in partially observable MDPs and apply it to suppress epileptic seizures on animal brain slices. Xu et al. [2017] apply locally linear reconstruction to approximate value functions and perform policy iteration. Antonova et al. [2020] propose an algorithm for learning the manifold structure with changing data distributions. Liu et al. [2021] propose encoding constraints of a robotic system (e.g. mechanical or safety) as a differentiable manifold and propose to learn policies that take actions along the tangent space of the said manifold.

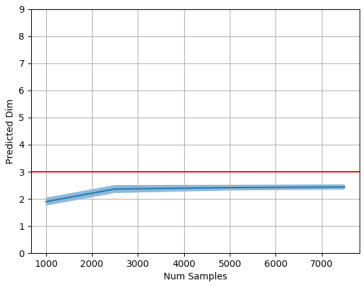
At the same time continuous RL has been applied to continuous robotic control [Doya, 2000, Deisenroth and Rasmussen, 2011, Duan et al., 2016]. We apply both continuous state and action, and continuous time RL as a theoretical model, that matches empirical observations, to study popular RL algorithms for the first time, to the best of our knowledge. Koditschek and Rimon [1990] provided an early results in the field, by showing that their construction of "sphere world" can be used to derive analytic solutions of control on analytic manifolds. Jenkins and Mataric [2004] extend the popular ISOMAP framework [Tenenbaum, 1997] to spatio-temporal data and they apply this extended framework to embed human motion data which has applications in robotic control. The work by Bayraktar and Kara [2022] is also deeply related as they propose a theoretically justified continuous



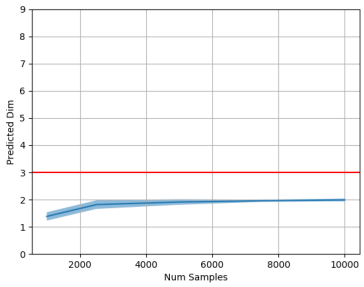
(a) Walker2D, $d_s = 16, d_a = 6$



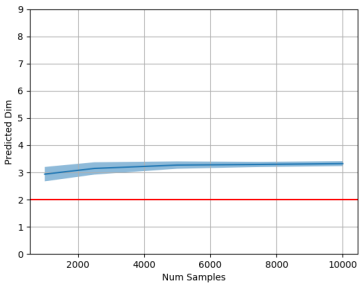
(b) Cheetah, $d_s = 16, d_a = 6$



(c) Gripper, $d_s = 11, d_a = 2$



(d) Swimmer, $d_s = 8, d_a = 2$



(e) IDP, $d_s = 11, d_a = 1$

Figure 3: We show that the estimate of the state manifold is close to $d_a + 1$ (horizontal red line) for various environments.

time algorithm for RL whilst modeling state trajectories as a diffusion process albeit in the Euclidean space.

Finally, researchers studying the theory of supervised deep learning have shown that the size of network required for generalising DNNs depends on the curvature and dimension of the manifold whilst depending weakly on the ambient data dimension [Shaham et al., 2015]. Schmidt-Hieber [2019] show that, under assumptions over the function being approximated, the statistical risk deep ReLU networks approximating a function can be bounded by an exponential function of the manifold dimension. This would suggest that the complexity of RL problems and data efficiency would be influenced more by the dimensionality of the state manifold as opposed to the ambient dimension.

7 Experiments on Dimensionality Estimates

To corroborate our main result (Theorem 3.2) we perform experiments in the MuJoCo domains provided in the OpenAI Gym [Brockman et al., 2016]. These are all continuous state and action

spaces with $d_a < d_s$ for simulated robotic control tasks. The states are, usually, sensor measurements such as angles, velocities or orientation, and the actions are torques provided at various joints. We estimate the dimensionality of the state manifold $\mathcal{S}_e$. To sample data from the manifold., we record the trajectories from multiple evaluation runs of the DDPG algorithm [Lillicrap et al., 2015] on MuJoCo environments with control inputs. We make two changes to this algorithm: we use GELU activation [Hendrycks and Gimpel, 2016] instead of ReLU, in both policy and value networks, and also use a single hidden layer network of width 400 instead of 2 hidden layers for both the networks. This is inline with the assumptions for Theorem 3.2 in Section 4.2. Performance is comparable to the original DDPG setting (see the Appendix E). We then randomly sample states from the evaluation trajectories to obtain a subsample of states, $\mathcal{D} = \{s_i\}_{i=1}^n \subset \mathcal{S}_e$.

We employ the dimensionality estimation algorithm introduced by Facco et al. [2017] to empirically corroborate Theorem 3.2. Facco et al. [2017] provide a method for estimating the intrinsic dimension of datasets characterized by non-uniform density and curvature. For a given dataset, $\mathcal{D}$, we compute the ratio of the two nearest neighbors $\mu_i = r_{i,2}/r_{i,1}$ where $r_{i,1}$ is the distance to the nearest neighbor in the dataset of s_i and $r_{i,2}$ is the distance to the second nearest neighbor. Facco et al. [2017] show that the logarithm of the probability distribution function of the ratio of the distances to two nearest neighbors is distributed inversely proportional to the degree of the intrinsic dimension of the data. Without loss of generality, we assume that $\{s_i\}_{i=1}^N$ are in the ascending order of r_i. We then fit a line going through the origin on the dataset $\{(\log(\mu_i), -\log(1 - i/N)\}_{i=1}^N$. The slope of this line is then the empirical estimate of $\dim(\mathcal{S}_e)$. We provide the results of this algorithm in Figure 3. For Cheetah and Inverted Double Pendulum we observe that the estimate overshoots but remains in the neighbourhood; this is also seen in Figure 4 by Facco et al. [2017] for the Isomap faces dataset. We attribute this slight overestimate to noise in the observations as is common in all implementation [Brockman et al., 2016]. Results of additional environments and further theoretical justification, as provided by Facco et al. [2017], are presented in Appendix F due to space constraints. We also provide additional experimental results in Appendix G for the same reason.

8 Discussion and Conclusion

We show that the dimensionality of the manifold is upper bounded by $d_a + 1$ and empirically verify it (Figure 3). This proves the popular manifold assumption [Mahadevan, 2005, Machado et al., 2017, 2018, Banijamali et al., 2018, Wu et al., 2019, Liu et al., 2021] holds under certain conditions in continuous-time reinforcement learning. It also shows that there is an underlying lower dimensional structure to the RL problem. Introducing the stochastic process formulation for DRL can lead to the application of the theory of stochastic processes to DRL. This formulation of RL over the randomness of initialisation is more relevant to practice because it allows us to study how the DRL agent performs under the distribution of initialisation instead of a fixed set of parameters. We also note that there have been a significant applications of diffusion processes in the field of unsupervised learning [Sohl-Dickstein et al., 2015, Ho et al., 2020, Song et al., 2021, Dhariwal and Nichol, 2021] and our work provides the theoretical grounds for a similar pursuit in RL.

9 Societal Impact

RL with has various practical applications [Mirhoseini et al., 2021, Jumper et al., 2021]. We try to provide theoretical reasoning for phenomena observed in RL. A strong theoretical backbone that does not stray too far from practice and can be verified empirically only contributes to advancement of the field. While we do recognize that the DRL is, unfortunately, resource intensive and consumes significant amounts of electricity, our experiments were minimal and did not employ vastly resource consuming hyper-parameter searches.

References

Rika Antonova, Maksim Maydanskiy, Danica Kragic, Sam Devlin, and Katja Hofmann. Analytic manifold learning: Unifying and evaluating representations for continuous control. *ArXiv*, abs/2006.08718, 2020.

Ershad Banijamali, Rui Shu, Mohammad Ghavamzadeh, Hung Hai Bui, and Ali Ghodsi. Robust locally-linear controllable embedding. In *AISTATS*, 2018.

Erhan Bayraktar and Ali Devran Kara. An approximate reinforcement learning algorithm for controlled diffusion processes. 2022.

Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *ArXiv*, abs/1606.01540, 2016.

Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Velivckovi'c. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *ArXiv*, abs/2104.13478, 2021.

Keith Bush and Joelle Pineau. Manifold embeddings for model-based reinforcement learning under partial observability. In *NIPS*, 2009.

G. Carlsson, T. Ishkhanov, V. D. Silva, and A. Zomorodian. On the local behavior of spaces of natural images. *International Journal of Computer Vision*, 76:1–12, 2007.

G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2:303–314, 1989.

Marc Peter Deisenroth and Carl Edward Rasmussen. Pilco: A model-based and data-efficient approach to policy search. In *ICML*, 2011.

Prafulla Dhariwal and Alex Nichol. Diffusion models beat gans on image synthesis. In *NeurIPS*, 2021.

Kenji Doya. Reinforcement learning in continuous time and space. *Neural Computation*, 12:219–245, 2000.

Yan Duan, Xi Chen, Rein Houthooft, John Schulman, and P. Abbeel. Benchmarking deep reinforcement learning for continuous control. In *ICML*, 2016.

Richard Durrett. Brownian motion and martingales in analysis. In *Brownian motion and martingales in analysis*, 1984.

Elena Facco, Maria d'Errico, Alex Rodriguez, and Alessandro Laio. Estimating the intrinsic dimension of datasets by a minimal neighborhood information. *Scientific Reports*, 7, 2017.

C. Fefferman, S. Mitter, and Hariharan Narayanan. Testing the manifold hypothesis. *arXiv: Statistics Theory*, 2013.

Yuri E. Gliklikh. Global and stochastic analysis with applications to mathematical physics. In *Global and Stochastic Analysis with Applications to Mathematical*, 2010.

Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv: Learning*, 2016.

Jonathan Ho, Ajay Jain, and P. Abbeel. Denoising diffusion probabilistic models. *ArXiv*, abs/2006.11239, 2020.

Elton P. Hsu. Stochastic analysis on manifolds. In *Stochastic analysis on manifolds*, 2002.

Odest Chadwicke Jenkins and Maja J. Mataric. A spatio-temporal extension to isomap nonlinear dimension reduction. In *ICML '04*, 2004.

John M. Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Zídek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A A Kohl, Andy Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David A. Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with alphafold. *Nature*, 596:583 – 589, 2021.

Daniel E. Koditschek and Elon D. Rimon. Robot navigation functions on manifolds with boundary. *Advances in Applied Mathematics*, 11:412–442, 1990.

Sergey Levine, Chelsea Finn, Trevor Darrell, and P. Abbeel. End-to-end training of deep visuomotor policies. *ArXiv*, abs/1504.00702, 2016.

Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Manfred Otto Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *CoRR*, abs/1509.02971, 2015.

Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Manfred Otto Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *CoRR*, abs/1509.02971, 2016.

Puze Liu, Davide Tateo, Haitham Bou-Ammar, and Jan Peters. Robot reinforcement learning on the constraint manifold. In *CoRL*, 2021.

Marlos C. Machado, Marc G. Bellemare, and Michael Bowling. A laplacian framework for option discovery in reinforcement learning. *ArXiv*, abs/1703.00956, 2017.

Marlos C. Machado, Clemens Rosenbaum, Xiaoxiao Guo, Miao Liu, Gerald Tesauro, and Murray Campbell. Eigenoption discovery through the deep successor representation. *ArXiv*, abs/1710.11089, 2018.

Sridhar Mahadevan. Proto-value functions: developmental reinforcement learning. *Proceedings of the 22nd international conference on Machine learning*, 2005.

Sridhar Mahadevan and Mauro Maggioni. Proto-value functions: A laplacian framework for learning representation and control in markov decision processes. *J. Mach. Learn. Res.*, 8:2169–2231, 2007.

Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim M. Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nazi, Jiwoo Pak, Andy Tong, Kavya Srinivasa, Will Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. A graph placement methodology for fast chip design. *Nature*, 594 7862:207–212, 2021.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin A. Riedmiller, Andreas Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236. URL `https://doi.org/10.1038/nature14236`.

John Nash. C1 isometric imbeddings. *Annals of Mathematics*, 60(3):383–396, 1954. ISSN 0003486X. URL `http://www.jstor.org/stable/1969840`.

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Johannes Schmidt-Hieber. Deep relu network approximation of functions on a manifold. *ArXiv*, abs/1908.00695, 2019.

John Schulman, S. Levine, P. Abbeel, Michael I. Jordan, and P. Moritz. Trust region policy optimization. *ArXiv*, abs/1502.05477, 2015.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *ArXiv*, abs/1707.06347, 2017.

Uri Shaham, Alexander Cloninger, and Ronald R. Coifman. Provable approximation properties for deep neural networks. *ArXiv*, abs/1509.07385, 2015.

David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, L. Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Vedavyas Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy P. Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529:484–489, 2016.

Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. *ArXiv*, abs/1503.03585, 2015.

Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *ArXiv*, abs/2011.13456, 2021.

Joshua B. Tenenbaum. Mapping a manifold of perceptual observations. In *NIPS*, 1997.

Yifan Wu, G. Tucker, and Ofir Nachum. The laplacian in rl: Learning representations with efficient approximations. *ArXiv*, abs/1810.04586, 2019.

Xin Xu, Zhenhua Huang, Lei Zuo, and Haibo He. Manifold-based reinforcement learning via locally linear reconstruction. *IEEE Transactions on Neural Networks and Learning Systems*, 28(4): 934–947, 2017. doi: 10.1109/TNNLS.2015.2505084.

Greg Yang and Edward J. Hu. Feature learning in infinite-width neural networks. *ArXiv*, abs/2011.14522, 2020.

Henry Zhu, Justin Yu, Abhishek Gupta, Dhruv Shah, Kristian Hartikainen, Avi Singh, Vikash Kumar, and Sergey Levine. The ingredients of real-world robotic reinforcement learning. *ArXiv*, abs/2004.12570, 2020.

Checklist

1. For all authors...
   (a) Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope? [Yes]
   (b) Did you describe the limitations of your work? [Yes] We discuss all the assumptions of our theoretical model.
   (c) Did you discuss any potential negative societal impacts of your work? [Yes] Section 9
   (d) Have you read the ethics review guidelines and ensured that your paper conforms to them? [Yes]
2. If you are including theoretical results...
   (a) Did you state the full set of assumptions of all theoretical results? [Yes] See Appendix A for a list and explanations
   (b) Did you include complete proofs of all theoretical results? [Yes]
3. If you ran experiments...
   (a) Did you include the code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL)? [Yes] See Appendix J
   (b) Did you specify all the training details (e.g., data splits, hyperparameters, how they were chosen)? [Yes] See experimental sections in the Appendix and main body
   (c) Did you report error bars (e.g., with respect to the random seed after running experiments multiple times)? [Yes]
   (d) Did you include the total amount of compute and the type of resources used (e.g., type of GPUs, internal cluster, or cloud provider)? [Yes]
4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets...
   (a) If your work uses existing assets, did you cite the creators? [Yes]
   (b) Did you mention the license of the assets? [Yes]
   (c) Did you include any new assets either in the supplemental material or as a URL? [No]
   (d) Did you discuss whether and how consent was obtained from people whose data you're using/curating? [N/A]
   (e) Did you discuss whether the data you are using/curating contains personally identifiable information or offensive content? [N/A]

5. If you used crowdsourcing or conducted research with human subjects...
   (a) Did you include the full text of instructions given to participants and screenshots, if applicable? [N/A]
   (b) Did you describe any potential participant risks, with links to Institutional Review Board (IRB) approvals, if applicable? [N/A]
   (c) Did you include the estimated hourly wage paid to participants and the total amount spent on participant compensation? [N/A]

A Assumptions

We list all the assumptions by section and consequently by theorems. We also provide some intuition on what these assumptions mean and how they limit our study.

Assumption made in Section 2.1 about continuous-time RL:

1. We assume that the transition function, f, is infinitely differentiable or smooth. This is done to ensure that the manifold it forms is also smooth. This might not always hold in practice but the transitions can be C^k smooth, meaning $k-$times differentiable.

We do not state the entirety of the assumptions made in Section 3 but we provide further intuition for them:

1. The no orbits assumption is strong but it is essential to proving Theorem 3.2. This might not hold strictly in practice but it can be argued that it holds almost always with probability 1, essentially the probability of an agent making an orbit is close to 0.
2. The action equivalence condition is a global rule on actions. This is once again required for proving Theorem 3.2. Intuitively, the idea is that if two actions have the same local effect on how the agents state changes in some state then it is also true everywhere in the state space. It is an assumption on the equivalence of assumptions and it helps construct a bijection in the proof of Theorem 3.2 since we can then choose actions that map uniquely at every state.

Assumptions made in section 4.2:

1. Local linearity of actions albeit restrictive only assumes actions mapping linearly onto the tangent space at a point. The linearity is for infinitesmally small time period ($t \to 0$).
2. We also assume that $\mathcal{A} = \mathbb{R}^{d_a}$ in the proof of Theorem 4.2.

B Proof of Theorem 3.2

We first prove it for the exact case where the dimensionality of the effective state space, $\mathcal{S}_e$, is $d_a + 1$. We do so under additional assumptions in the following proposition.

Proposition B.1. *Under the assumptions of Section 2.1 and 3 with the added assumption that the transition function, f, is injective in $\mathcal{A} \times [-1, 1]$ for fixed $s \in \mathcal{S}$ the dimensionality of the effective state space, $\mathcal{S}_e \subset \mathbb{R}^{d_s}$, is $d_a + 1$.*

Proof. Given any state $s \in \mathcal{S}_e$ we can construct an open neighbourhood that maps to an open subset of $\mathbb{R}^{d_a+1}$. Since we know that $s \in \mathcal{S}_e$ there exists a state $s' \in \mathcal{S}_e$ such that $f(s', a, \epsilon) = s$ for some $a \in \mathcal{A}$ and $\epsilon > 0$. Now, for a fixed s' consider the map $\psi_{s'} : \mathcal{A} \times (\epsilon - \eta, \epsilon + \eta) \to \mathcal{S}_e$ such that $\psi_{s'}(a, t) = f(s', a, t)$ where $0 < \eta < \epsilon$ and $a \in \mathcal{A}$.

We need to show that this map, $\psi_{s'}$, is a diffeomorphism and maps to an open subset of S_e. Since the transition function, f, is smooth we have that $\psi_{s'}$ is also smooth in its range. Also note that f is injective $\mathcal{A} \times [-1, 1]$ for fixed s' and therefore it maps $\mathcal{A} \times (\epsilon - \eta, \epsilon + \eta)$ uniquely to a set U. This set U is a subset of $\mathcal{S}_e$ by definition. Note that U is open because $\mathcal{A} \times (\epsilon - \eta, \epsilon + \eta)$ is open by the invariance of domain. This leads to the conclusion that $\psi_{s'} : \mathcal{A} \times (\epsilon - \eta, \epsilon + \eta) \to U \subset \mathcal{S}_e$ is a smooth bijection. Thereby providing us a smooth parametrization from $\mathbb{R}^{d_a+1}$ to U. Its inverse, $\psi_{s'}^{-1}$, is the coordinate chart that is smooth everywhere except where the $\nabla_s \psi_{s'}$ is non zero.

We can similarly construct these diffeomorphisms from $\mathbb{R}^{d_a+1}$ to $U \subset \mathcal{S}_e$ for all $s \in \mathcal{S}_e$. Finally, by Definition 2.1 we have that $\mathcal{S}_e$ is a $d_a + 1$ dimensional manifold.

□

Now we prove the general case without the assumption of injectivity on f. To do so we construct an injection $f' : \mathcal{S} \times \mathcal{A}' \times \mathbb{R}^+ \to \mathcal{S}$ where $\mathcal{A}' \subset \mathbb{R}^{d'}$ is an open set such that $d' \leq d_a$. We make two additional assumptions. The first one being that for any policy $\pi \in \Pi$ we do not have any critical points or loops in the differential field defined by g_π. Formally, we have the following conditions

$g_\pi(s) \neq \mathbf{0}$ for all $s \in \mathcal{S}$ and for a fixed policy π the solution for the equation $s_t^\pi = s$, if it exists, is unique in t. The second one is that if two actions have the same outcome at one state then they have the same outcome at all states. More formally, there exists an $r > 0$ such that for all $0 < \epsilon < r$, if we have $f(s, a_1, \epsilon) = f(s, a_2, \epsilon)$ for some fixed $a_1, a_2 \in \mathcal{A}$ and any one $s \in \mathcal{S}$ then it holds for all $\mathcal{S}$.

Under the assumptions of Section 2.1 and the ones stated above we restate the main theorem below before presenting the proof.

Theorem 3.2. *The set of effective states, $\mathcal{S}_e$, is a smooth manifold of dimensionality at most $d_a + 1$.*

Proof. For every a_α and a_β such that $f(s, a_\alpha, \epsilon) = f(s, a_\beta, \epsilon)$ we create a map $h : \mathcal{A} \to \mathcal{A}' \subset \mathcal{A}$ such that $h(a_\alpha) = a_\beta$. Further, we choose h such that it is continuous and smooth. Note that such an h exists because f is continuous and smooth. Now consider a function $f' : \mathcal{S}_e \times \mathcal{A}' \times (\epsilon - \eta, \epsilon + \eta)$ which is a restriction of f to the domain $\mathcal{S}_e \times \mathcal{A}' \times (\epsilon - \eta, \epsilon + \eta)$. Now for a fixed s we can construct an open neighbourhood using $\psi_{s'} : \mathcal{A}' \times (\epsilon - \eta, \epsilon + \eta)$ constructed as in Proposition B.1, $\psi_{s'}(a, t) = f'(s', a, t)$ such that $\psi_{s'}(a, \epsilon) = s$ for some $a \in \mathcal{A}'$ and $\epsilon > 0$. Now we know that this is a bijection from assumptions 1 and 2 in Section 3.1 as long as we choose ϵ and η small enough to satisfy assumption 2. The construction of set $\mathcal{A}'$ ensures that each action, for a fixed time t, maps to a fixed state. Further, the assumption on no orbits ensures that for a fixed action $a \in \mathcal{A}'$ there is a unique solution to $f(s', a, t) = s''$ for a fixed a, s' and s'' in t. Therefore, we constructed a smooth bijection $\psi_{s'}$ which maps an open set $\mathcal{A}' \times (\epsilon - \eta, \epsilon + \eta)$ to an open subset of $\mathcal{S}_e$ for every $s \in matS_e$.

Finally, we conclude by stating that the dimensionality of the manifold is equal to the of $\dim(\mathcal{A}') + 1$ which is less than or equal to $d_a + 1$. □

Finally, we prove Corollary 3.3. The proof is fairly straightforward following Definition 2.2 and Definition 3.1. Since the manifold $\mathcal{S}_e$ is defined as the union of all continuous curves from all possible smooth policies we can find a curve such that for any $s \in \mathcal{S}_e$ and $a \in \mathcal{A}$ for some policy $\pi(s) = a$ in the neighborhood of s $\mathcal{S}_e \cap B(s, \epsilon)$ where $B(s, \epsilon)$ is a Euclidean ball of radius $\epsilon > 0$ centered at s. This would mean that for this policy, starting at s_0, there is a curve γ such that for some time t and some interval $\eta > 0$ we have $\gamma(t) = s$ and therefore $\dot{\gamma}(t) = v$ where $v \in T_s\mathcal{S}_e$. This vector v can now be uniquely mapped to the action and therefore can be labeled v_a as in Corollary 3.3. We also note that this vector is unique independent of the choice of ϵ or η.

C Additional Background on Stochastic Processes on Manifolds

We omit the definition of a probability space in the main body of the paper in the interest of succinctness. We define a probability space $(\Omega, \mathcal{F}, P)$, where each one of the symbols is defined as:

1. Ω is the sample space, for example it could be the set of all smooth policies Π,
2. $\mathcal{F}$ is a set of events, a single event could be a set of policies from Π,
3. P is the probability function that maps an event to the probability of it occurring, for example it could be the probability of a set of policies being selected by the agent, which is a subset of Π.

While we use the Stratonovich integral the most basic building block of stochastic integrals is the Itô integral. Following the notation in Section 4.1, we now define the Itô integral which is applied to integrate random variables that vary with time. Let A be a random linear operator going from $\mathbb{R}^k$ to $\mathbb{R}^n$. For a partition of the time interval $q = (0 = t_0 < t_1 < t_2 ... < t_q = T)$ consider the integral sum

$$\sum_{i=0}^{q-1} A_{t_{i+1}}(w(t_{i+1}) - w(t_i)),$$

where $w(t)$ is a Wiener process in $\mathbb{R}^k$. As $\text{diam}(q) \to 0$ the limiting sum is termed the Itô integral. There is a fundamental difference between the two integrals Itô and Stratonovich in how they approximate the linear operator, A. For Itô integral the linear operator between time t_i and t_{i+1} is approximated by the linear operator at end of time interval: $A_{t_{i+1}}$. As opposed to the average of the linear operator at the start and end interval: $(A_{t_{i+1}} + A_{t_i})/2$. The reason we use Stratonovich

is because it ports directly to the case of stochastic process on the manifold from that of Euclidean space. Let A be a random linear operator from $\mathbb{R}^k to \mathbb{R}^n$, then for the Euclidean case the formula for converting from Strantonovich to Itô integral is:

$$\int_0^t A(l) \circ dw_k(l) = \int_0^t A(l) dw_k(l) + \frac{1}{2} \int_0^t dA(l) \circ dw_k(l),$$

where both the integral on the right hand side are of the Itô form and the one on the left is Strantonovich.

D Proof of Theorem 4.2

We first state the law of large numbers.

We first state the theorem as in the main body of the paper.

Theorem 4.2. *Let S_t denote the state random variable at time t in a continuous time MDP for a randomly initialised, as in the maximal update parametrization, policy with single hidden layer fully connected DNN of width n with GELU activation and additive Gaussian noise. Let the continuous time MDP have a start state s_0, action bundle g that is linear in a. The stochastic differential equation for S_t, in Stratonovich formulation,*

$$dS_t = b_s dt + A_s \circ dw_{d_a}(t),$$

where b_s and A_s are as in Equation 3, converges to a unique solution in distribution as $n \to \infty$

$$S_t = s_0 + \int_0^t b_s dl + \int_0^t A_s \circ dw_{d_a}(l), \tag{5}$$

where the second integral is a Stratonovich integral. Additionally, it is a diffusion process on the manifold $\mathcal{S}_e$.

Proof. The policy network is parametrized by a single layer for any input $x \in \mathbb{R}^{n_{\text{in}}}$ we have

$$z(x) = Ux, \;\; h(x) = \varphi(z(x)), \;\; F(x) = Vh(x),$$

where W is an $n_{\text{in}} \times n$ matrix, φ is GELU activation and V is an $n \times n_{\text{out}}$ matrix. This neural network is initialized randomly as done in the *maximal update parametrization* (MUP) introduced by Yang and Hu [2020]: $U = \sqrt{n}u, V = \frac{v}{\sqrt{n}}$ and with initialization $u_{i,j}, v_{i,j} \sim \mathcal{N}(0, 1/n)$.

We first show that as the width of the DNN goes to infinity $n \to \infty$ the output is deterministically 0, as done in Section 6.1 by Yang and Hu [2020]. We represent by Z^U the distribution of individual entries of the matrix U, specifically $Z^U \sim \mathcal{N}(0, 1)$. Similarly, $Z^{nV} \sim \mathcal{N}(0, 1)$. For a fixed input $x \in \mathbb{R}^{n_{\text{in}}}$ the inputs of the first layer are independent and identically distributed iid $Z^{z_i} = \sum_{j=1}^{n_{\text{in}}} x_j Z^U_{i,j}$. Therefore, for the first hidden layers' output we also have iid outputs $Z^{h_i} = \varphi(Z^{z_i})$. Finally, the output $F(x)_i = \sum_{j=1}^{n} nV_{i,j} h_j \to 0$ as $n \to 0$ this follows from law of large numbers. Intuitively, the sum of independent random variables converges to its mean which in this case is 0. There is an Ornstein Uhlenbeck process with mean 0 and variance σ_a added to the output [Lillicrap et al., 2015], denoting the random variable for action a_t at time t, the output of the policy is, for a DNN of width n,

$$a_t = F(s_t) + \sigma dw_{d_a}(t),$$

where $dw_{d_a}(t)$ is a d_a dimensional Wiener process and s_t the random variable corresponding to the state at time t. Therefore as $n \to \infty$ we have a_t converges in distribution to $\sigma dw_{d_a}(t)$. As per our assumption we have that the action bundle is linear for a fixed state, $g(s, \cdot)$. Therefore, for a state s at time t we have the stochastic differential equation,

$$dS_t = g_s(a_t) = b_s dt + A_s \circ dw_{d_a}(t), \tag{6}$$

where b_s and A_s are as defined in section 4.2.

Finally we need to show a unique solution exists for the stochastic differential equation 6. We first note that since g is smooth in state and action jointly we can claim that it is locally Lipschitz is (s, a) jointly, therefore b_s and A_s are locally Lipschitz. Therefore, by Theorem 1.1.10 in the textbook by Hsu [2002] we have that the solution is unique. □

Note that here the action set is assumed to be the entirety of $\mathbb{R}^{d_a}$. This is not the case for RL environments where the actions are limited to some subset of $\mathbb{R}^{d_a}$.

E DDPG modified architecture comparison

We provide the comparison between single hidden layer network and multiple hidden layer network because our results in section 4 are for single hidden layer. The same architecture is used by Lillicrap et al. [2015] for the policy and value function DNNs which is two hidden layers of width 300 and 400 with ReLU activation. Here we provide the comparison to single hidden layer width 400 and MUP [Yang and Hu, 2020] with GELU activation for the architecture used by Lillicrap et al. [2015]. We provide this comparison in Figure 4 and note that the performance remains comparabale for both the architectures. All results are averaged over 6 runs over different seeds. We use a PyTorch based implementation for DDPG with modifications for MUP parametrisation and the use of GELU units. The base implementation of the DDPG algorithm can be found here: https://github.com/rail-berkeley/rlkit/blob/master/examples/ddpg.py. The hyperparameters are as in the base implementation.

F Dimensionality Estimation as in Facco et al. [2017]

We provide further justification for the algorithm we use as described in Section 7. Since we are introducing the work by Facco et al. [2017] for manifold dimensionality estimation to RL we provide further background on the method. Let the dataset be randomly sampled points from a manifold M embedded in $\mathbb{R}^d$ denoted by $\mathcal{D} = \{x_i\}_{i=1}^N$. For a point x_i from the dataset $\mathcal{D}$ let $r_{i,1}, r_{i,2}, r_{i,3}, ...$ be a sorted list of distances of other points in the dataset from x_i and they set $r_0 = 0$. The volume of the hyperspherical shell enclosed between two successive neighbors $k-1$ and k is given by $\delta v_k = \omega_d(r_k^d - r_{k-1}^d)$, where ω_g is the volume of the $d-$sphere with unitary radius. Facco et al. [2017] prove in their supplementary material if the density is constant around a point then all the δv_k are drawn independently from an exponential distribution with rate equal to density ρ:

$$\Pr(\delta v_k \in [v, v + dv]) = \rho e^{-\rho v} dv.$$

This gives the probability density function for the ratio for volumes of two successive shells $v_{\text{ratio}} = \frac{\delta v_1}{\delta v_2}$ is $\frac{1}{(1+v_{\text{ratio}})^2}$. Let the ratio of distances to the two nearest neighbors be $\mu = \frac{r_2}{r_1}$ which is related to v_{ratio} is $v_{\text{ratio}} = \mu^d - 1$. The cumulative distribution (cdf) for μ is then obtained to be

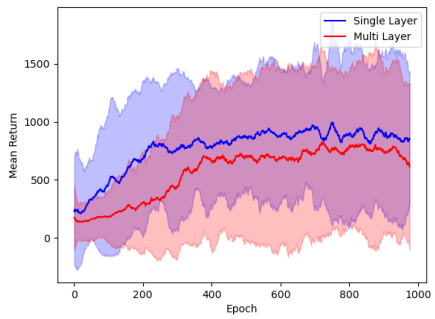
$$\text{cdf}(\mu) = (1 - \mu^{-m}) 1_{[1,\infty)}(\mu),$$

where $1_{[1,\infty)}$ is the indicator function for the set $[1, \infty)$ and $m = \dim(M)$. Therefore, it depends explicitly on the dimension of the manifold M. This means the dimension of the manifold M can be estimated as

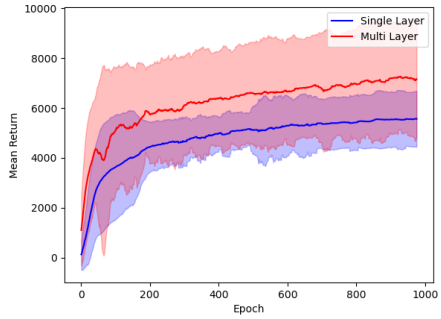
$$m = \frac{-\log(1 - \text{cdf}(\mu))}{\log(\mu)}. \tag{7}$$

Note that the density ρ does not appear in the equation and therefore is not susceptible to variations in the probability density variation across points. This is especially relevant in RL where the states are sampled by the agent choosing random actions to traverse the effective state manifold $\mathcal{S}_e$ and the states sampled are restricted by the action set available to the agent.

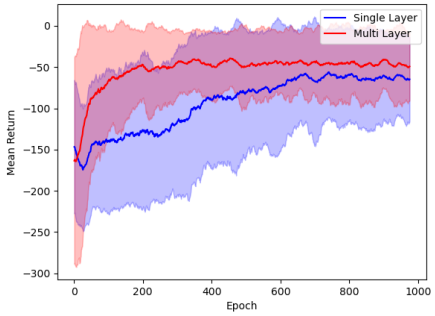
In light of the above observations and for the sake of completeness, we restate the algorithm presented by Facco et al. [2017] for estimating intrinsic data dimension (or dimension of the manifold M) from Section 7.



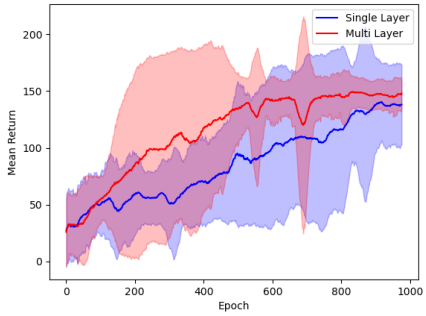
(a) Walker2D



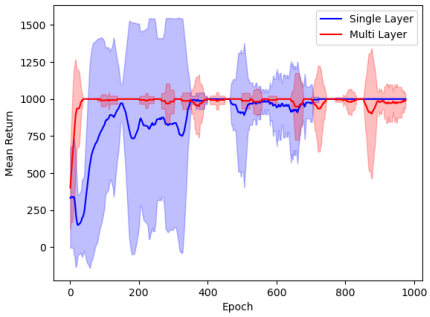
(b) Cheetah



(c) Reacher



(d) Swimmer



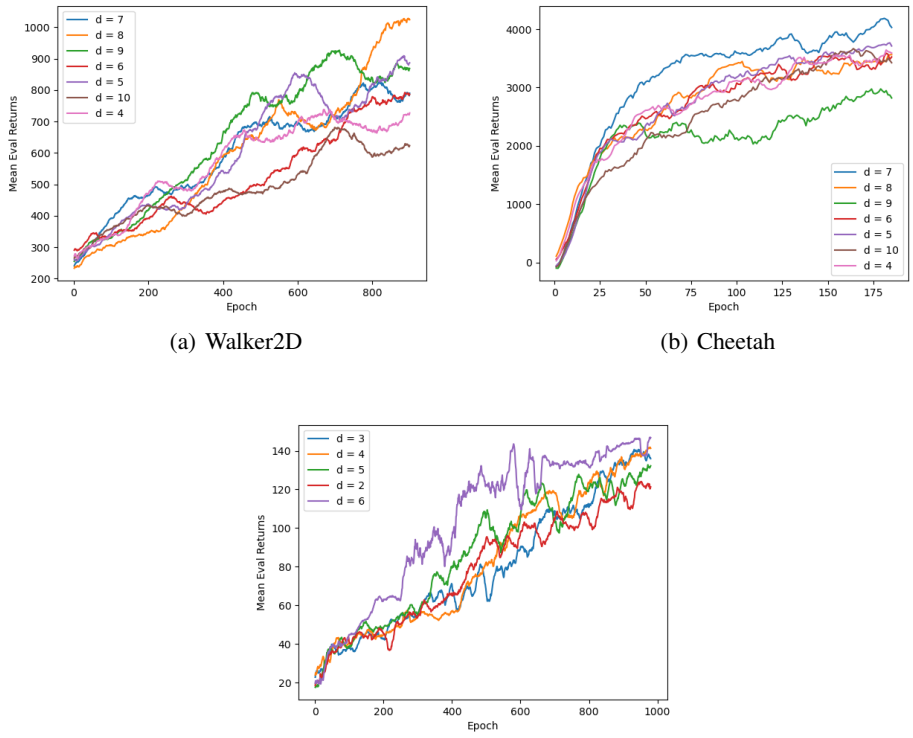
(e) Inverted Double Pendulum

Figure 4: Comparison of single hidden layer (blue) and multiple hidden layer (red) architectures for DNNs.

1. Compute $r_{i,1}$ and $r_{i,2}$ for all data points i.
2. Compute the ratio of the two nearest neighbors $\mu_i = r_{i,2}/r_{i,1}$.
3. Without loss of generality, given that all the points in the dataset are sorted in ascending order of μ_i the empirical measure of cdf is i/N.
4. We then get the dataset $\mathcal{S} = \{(\log(\mu_i), -\log(1 - i/N)\}$ through which a straight line passing through the origin is fit.

The slope of the line fitted as above is then the estimate of the dimensionality of the manifold.

G Additional Experiments



(a) Walker2D

(b) Cheetah

(c) Swimmer

Figure 5: Comparison of varying width of the second hidden layer architectures for DNNs. All evaluation returns are smoothend over 25 epochs.

We present the results of additional experiments we performed. We modify the architecture for the DNNs policy and value function. We introduce an additional hidden layer and therefore the architecture we use has 400 units, the second layer has varying width which we label as d and the third hidden layer has 400 units. We use GELU activation everywhere. The reason for this choice is to project the input down to lower dimensions which is close to the dimensionality of the state manifold $\mathcal{S}_e$ and we evaluate the performance of varying the width at layer 2. The results are in Figure 5.

We note that the performance peaks for $d = \dim(\mathcal{S}_e)$ only in case of Cheetah and it is worse for higher and lower values of w_2. This pattern does not hold for Walker2D and Swimmer environments. Our hypothesis is that the projection down to lower dimensions being learned by the DNNs is neither isometric nor diffeomorphic. By an isometric embedding we mean that the distance between any two points on the manifold is the same as in the embedded space. In addition to this, although any Reimannian manifold can be embedded isometrically into an Eucldiean space but not necessarily of

the same dimensionality of the manifold. The Nash embedding theorem states that the a C^1 manifold can be embedded into a Euclidean space of dimensionality at most $m(m+1)$ or $(3m+11)/2$ [Nash, 1954]. Therefore we do not know the optimal embedding dimension for the state manifold $\mathcal{S}_e$ and it is subject to future studies.

H Compute Requirements

For each one of our experiments we perform 6 different runs with varying seeds to obtain the results. Since experiments in Section 7 only require trajectories we reuse the trajectories sampled form the ReLU and GELU comparison experiments in Appendix E. For all the experiments in Appendix E we required approximately 180 hours of processing time on NVIDIA GeForce RTX 3090 GPUs which have 24 GB of RAM. We also run two runs of the DDPG algorithm simultaneously on each GPU, owing to the 24 GB RAM availability per GPU, which cuts our run time into half.

I Steps for Running the Code

All code is in python and was tested for python 3.6. We provide the implementation of DDPG using GELU units and MUP parametrization in the `code/rlkit/` folder of our supplementary material To install the environment we recommend installation using the `setupy.py` file present in the folder. For running the experiment, there are multiple files in the `rlkit/examples/ddpg*.py`, each one uses a different architecture and environment details of which can be found within. To execute the code, e.g. for `ddpg.py`, run the following command from the rlkit folder:

```
python examples/ddpgy.py 115 gelu
```

where "115" is the seed and "gelu" tells the program to use gelu activations. Our code samples trajectories and the location of the folder can be specified in the file `path_collector.py`

The code for dimensionality estimation using neighborhood data is fairly simple and only requires version 1.1.1 of the python package `scikit-learn` [Pedregosa et al., 2011]. It is provided in `code/data-processing/dim-estimate`. The main file is `dim-estimate.py`. It expects a pickle file which is an array of all states sampled for an agent. The code for cleaning the samples, from runs of the DDPG code as described above, and acquiring them in the required format can be found in the file `run_data_processing.py`.